

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3084 - Data Science

Erick Marroquín



Laboratorio 8 - DuckDB

**Análisis de viajes de taxi de Nueva York con Parquet,
DuckDB y Metabase**

Ricardo Godínez - 23247

Vianka Castro - 23201

Sebastián Bustamante - 22291

GUATEMALA, 6 de octubre de 2026

Índice

1. Introducción	2
2. Objetivos	2
Objetivo general	2
Objetivos específicos	2
3. Desarrollo.....	3
Preparación del ambiente.....	3
Sistema de descarga	3
Consultas directas sobre Parquet	4
Análisis exploratorio	4
Incorporación de 2024 y benchmark.....	5
Indicadores y tablero	5
Incorporación de 2025	6
4. Resultados	6
Exploración inicial de 2026	6
Análisis exploratorio	7
Consulta conjunta de 2024 y 2026	8
Parquet contra tabla DuckDB	8
Indicadores en Metabase	9
Evolución 2024 a 2026	14
5. Discusión	15
Parquet o tabla materializada.....	15
Frente a Pandas	16
Diseño para nuevos datos y reproducibilidad	16
Limitaciones.....	16
6. Conclusiones	17
7. Referencias.....	17

1. Introducción

En este laboratorio trabajamos con los registros de viajes de taxi que publica la Comisión de Taxis y Limusinas de Nueva York (TLC). Cada mes la TLC publica un archivo por servicio, y nosotros usamos los taxis amarillos (Yellow Taxi) y verdes (Green Taxi) de 2024, 2025 y 2026. Al final del laboratorio el conjunto llegó a 64 archivos y más de 121 millones de viajes, un volumen que ya no cabe cómodamente en la memoria de una computadora personal si se carga con herramientas como Pandas.

La herramienta central fue DuckDB (un motor de base de datos analítico que corre dentro del mismo proceso de Python, sin servidor aparte, parecido a SQLite pero pensado para consultas que recorren millones de filas). DuckDB puede consultar directamente archivos Parquet (un formato de archivo que guarda los datos por columna y comprimidos, de modo que una consulta que solo usa la tarifa no tiene que leer las otras diecinueve columnas). Esto nos permitió analizar todo el conjunto con SQL sin un paso previo de carga.

El trabajo avanzó en etapas: preparamos un ambiente reproducible con Docker, completamos el sistema de descarga, exploramos los datos de 2026, incorporamos 2024, comparamos consultar Parquet contra una tabla materializada, construimos diez indicadores en un tablero de Metabase y, al final, agregamos 2025 para analizar la evolución de tres años. En el camino aprendimos que con datos grandes no basta con que la consulta funcione: también hay que revisar si los datos cambiaron su forma de registrarse con el tiempo.

2. Objetivos

Objetivo general

Construir un flujo reproducible para descargar, consultar y analizar los viajes de taxi de Nueva York con DuckDB, capaz de incorporar nuevos años con cambios mínimos.

Objetivos específicos

1. Preparar un ambiente con Docker Compose que incluya Python, DuckDB, JupyterLab y Metabase.
2. Completar un sistema de descarga que obtenga los archivos de la fuente original sin repetir los que ya existen.

3. Explorar los archivos Parquet con consultas directas e identificar problemas de calidad de datos.
4. Comparar el rendimiento de consultar Parquet directamente contra una tabla materializada en DuckDB.
5. Diseñar al menos seis indicadores respaldados por SQL y visualizarlos en un tablero de Metabase.
6. Incorporar 2024 y 2025 y analizar cómo cambian los indicadores entre 2024, 2025 y 2026.

3. Desarrollo

Preparación del ambiente

Hicimos un fork del repositorio del curso, lo clonamos y levantamos el ambiente con Docker Compose, que construye las imágenes y arranca los servicios en segundo plano. Quedaron dos servicios: `lab8-lab`, con Python 3.11, DuckDB, JupyterLab, Pandas, PyArrow, Matplotlib y Requests (en el puerto 8888), y `lab8-metabase`, con Metabase y el driver de DuckDB ya instalado (en el puerto 3000). Ambos contenedores montan la carpeta `data/` del proyecto en `/workspace/data`, así que el archivo que crea Python es el mismo que lee Metabase.

Las versiones están fijadas en `requirements.txt` y en las imágenes de Docker. Esto importa más de lo que parece: la versión de DuckDB en Python tiene que coincidir con la del driver de Metabase para que ambos lean la misma base, y cualquier integrante o revisor obtiene las mismas cifras ejecutando los mismos comandos, sin depender de lo que tenga instalado en su computadora.

Sistema de descarga

El script `scripts/download_data.py` que recibimos ya recorría los doce meses, permitía elegir Yellow o Green, evitaba repetir archivos y descargaba sobre un archivo temporal. Al revisarlo notamos que consideraba válido cualquier archivo con tamaño mayor que cero, lo que podía aceptar una descarga interrumpida. Agregamos la función `es_parquet_valido`, que revisa la firma `PAR1` con la que todo Parquet empieza y termina, y la usamos tanto para decidir si un archivo existente se omite como para validar el archivo temporal antes de renombrarlo.

```
def es_parquet_valido(ruta: Path) -> bool:
    with ruta.open("rb") as archivo:
        inicio = archivo.read(4)
        archivo.seek(-4, 2)
```

```
final = archivo.read(4)
return inicio == b"PAR1" and final == b"PAR1"
```

Antes de descargar, el script pregunta al servidor si el mes ya está publicado (con una petición HEAD, que consulta si el archivo existe sin bajarlo). Así distinguimos un mes que todavía no publica la TLC de una descarga fallida. Para comprobar que la descarga de 2026 estaba completa hicimos tres verificaciones: comparamos los archivos locales con los publicados (enero a agosto, 8 por servicio), corrimos el descargador por segunda vez y obtuvimos 0 descargados y 16 existentes, y comprobamos que DuckDB podía abrir todos los archivos. Más adelante el script también descarga la tabla de zonas de la TLC, que traduce los identificadores de zona a barrio y nombre.

Consultas directas sobre Parquet

Para la exploración inicial consultamos los archivos sin importarlos a ninguna tabla. Todas las consultas quedaron en `sql/01_exploracion_inicial.sql`, cada una con comentarios que indican su objetivo, los archivos fuente y la decisión que tomamos a partir del resultado. Un script las ejecuta y genera la documentación con los resultados, de modo que al llegar un mes nuevo basta con volver a correrlo.

```
SELECT count(*)
FROM read_parquet('data/raw/*/2026/*.parquet',
                  filename = true, union_by_name = true);
```

Desde esta etapa encontramos que Yellow y Green no tienen el mismo esquema. Yellow llama a las fechas `tpep_pickup_datetime` y `tpep_dropoff_datetime`, Green usa `lpep_*`, Green tiene `ehail_fee` y `trip_type` y Yellow tiene `Airport_fee`. Por eso usamos `union_by_name = true` (que une las columnas por nombre y no por posición) y normalizamos las fechas dentro de cada consulta. También decidimos no eliminar registros de forma automática: cada consulta posterior declara qué filtro aplica y por qué, porque un monto negativo puede ser un reembolso mientras una fecha final anterior a la inicial parece un error.

Análisis exploratorio

Planteamos ocho preguntas sobre 2026, guardadas en `sql/02_analisis_exploratorio.sql` con su justificación: evolución mensual, patrón por hora y tipo de día, características por servicio, formas de pago, propinas con tarjeta, percentiles, valores atípicos y rutas frecuentes. Como Yellow tiene más del 98 % de los viajes, comparamos los servicios con porcentajes y medianas en lugar

de cantidades absolutas, y restringimos las propinas a pagos con tarjeta, porque la TLC solo registra de forma automática la propina de tarjeta.

Incorporación de 2024 y benchmark

Para incorporar 2024 bastó con pedir ese año al descargador: los archivos de 2026 se conservaron y no se descargaron de nuevo. Las consultas siguieron funcionando cambiando el patrón `data/raw/<tipo>/2026/*.parquet` por `data/raw/<tipo>/**/*.parquet` y sacando el año del nombre del archivo con `regexp_extract`.

Después comparamos dos estrategias: consultar Parquet directamente y materializar los datos en una tabla DuckDB con `CREATE TABLE ... AS SELECT`. Elegimos tres consultas representativas (un resumen mensual, una distribución de pagos de baja cardinalidad y un ranking de rutas con función de ventana), las ejecutamos sobre ambas fuentes con 2024, 2026 y ambos años juntos, y registramos por separado la primera ejecución y la mediana de las siguientes, porque la primera incluye el calentamiento de páginas y metadatos.

Indicadores y tablero

Para el tablero decidimos materializar, siguiendo lo que nos enseñó el benchmark: Metabase repite las mismas consultas cada vez que alguien abre el tablero. El script `scripts/build_indicadores.py` ejecuta `sql/05_modelo_indicadores.sql`, que crea tres objetos en `data/processed/taxi.duckdb`: la tabla `viajes`, que une Yellow y Green de todos los años con nombres comunes; la tabla `zonas`, a partir del CSV de la TLC; y la vista `viajes_validos`, que concentra en un solo lugar el criterio de viaje plausible.

```
CREATE OR REPLACE VIEW viajes_validos AS
SELECT * FROM viajes
WHERE year(pickup_datetime) = anio
      AND duracion_min > 0 AND duracion_min <= 360
      AND trip_distance > 0
      AND total_amount >= 0;
```

Los diez indicadores viven en `sql/06_indicadores.sql`. Además de la pregunta, la justificación y la interpretación, cada consulta lleva en sus comentarios el tipo de gráfico y los ejes. Con eso escribimos `scripts/setup_metabase.py`, que usa la API de Metabase para conectar la base en modo solo lectura, crear una pregunta SQL por indicador y acomodarlas en el tablero. Así el tablero no se arma a mano y se puede regenerar cuando cambian los datos.

Al construir los indicadores los datos nos obligaron a cambiar dos preguntas. Queríamos ver si el origen de la solicitud (`request_source`, que distingue viajes pedidos por plataformas como Uber o Lyft) se relacionaba con la propina y con el tipo de tarifa. Al revisar, encontramos que esa columna solo existe desde junio de 2026 y que en los viajes por plataforma la propina llega como cero y `RatecodeID` viene siempre vacío. Reemplazamos esas preguntas por la propina según la distancia y el uso y costo de cada tipo de tarifa. También cambiamos promedios por medianas en el indicador de origen, porque los promedios de distancia de las plataformas estaban inflados por valores extremos (Uber tenía 12.1 millas de promedio, pero 3.0 de mediana).

Incorporación de 2025

Para el último año cambiamos una sola línea del descargador, `ANIOS_PREDETERMINADOS = (2024, 2025, 2026)`. La primera ejecución descargó 24 archivos y omitió 40, y la segunda reportó 0 descargados y 64 existentes. Reconstruimos la base con 121 millones de viajes y volvimos a ejecutar todos los notebooks sin modificar ninguna consulta. También agregamos una verificación que compara los registros de los Parquet con los de la tabla, y otra que lista las columnas que no aparecen en todos los archivos.

En esta etapa tuvimos dos problemas prácticos. El archivo de la base llegó a 5.9 GB porque `CREATE OR REPLACE` no devuelve al disco el espacio de la tabla anterior, así que hicimos que el script recree la base desde cero (quedó en 3.7 GB), no sin antes comprobar que ningún otro proceso la tenga abierta. Además, una consulta con varias medianas exactas sobre unos 90 millones de filas agotó la memoria del contenedor; la resolvimos limitando DuckDB a 3 GB, con lo que usa disco temporal en lugar de fallar.

4. Resultados

Exploración inicial de 2026

La exploración encontró 16 archivos (enero a agosto de 2026 para cada servicio) y ningún mes vacío. La Tabla 1 muestra la cantidad de registros por servicio.

Tabla 1. Registros disponibles en 2026 (enero a agosto).

Servicio	Registros
Yellow Taxi	29,703,355
Green Taxi	337,114

Servicio	Registros
Total	30,040,469

La revisión de calidad mostró 31 viajes con fecha de inicio fuera de 2026, 15 con fecha final anterior a la inicial, 7,765,463 registros sin cantidad de pasajeros, 964,443 viajes con distancia igual o menor que cero y 162,858 con monto total negativo. Con esto confirmamos que un archivo legible no necesariamente tiene sus variables completas.

Análisis exploratorio

La Tabla 2 resume las características de los viajes plausibles de cada servicio en 2026.

Tabla 2. Características de los viajes plausibles por servicio, 2026.

Servicio	Distancia mediana (mi)	Duración mediana (min)	Monto mediano (USD)	Pasajeros promedio
Yellow	1.92	13.95	23.58	1.25
Green	2.15	13.35	20.52	1.30

El patrón por hora de la Figura 1 fue uno de los hallazgos más claros. En días laborales hay un pico por la mañana y el máximo llega entre las 17 y las 18 horas (7.37 % de los viajes laborales de Yellow a las 18:00 y 8.08 % de los de Green a las 17:00). Los fines de semana no tienen pico matutino y la madrugada pesa mucho más.

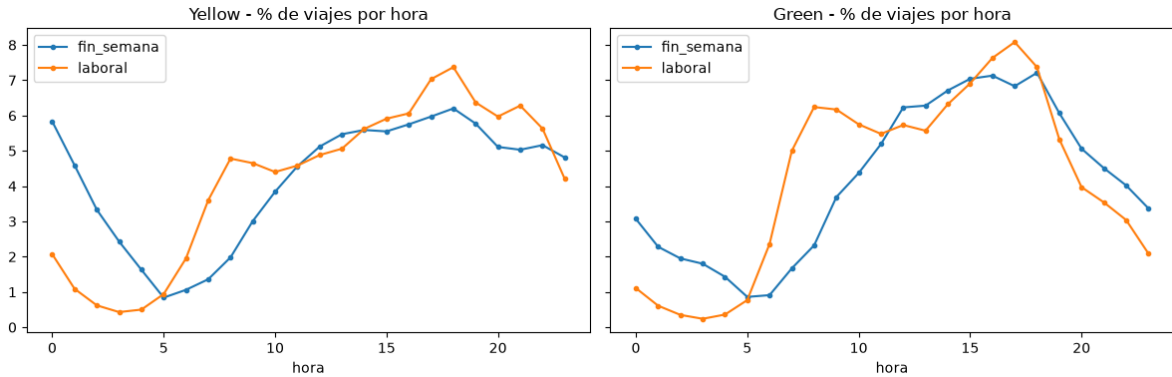


Figura 1. Porcentaje de viajes por hora en días laborales y fines de semana, 2026.

En los pagos con tarjeta, 91.10 % de los viajes Yellow y 90.45 % de los Green registran propina, con medianas de USD 3.29 y USD 3.08. La tarjeta es la forma de pago más frecuente en ambos servicios (63.77 % en Yellow y 65.25 % en Green).

Consulta conjunta de 2024 y 2026

Al incorporar 2024 obtuvimos los doce meses de cada servicio y una sola consulta pudo leer los dos años, como muestra la Tabla 3.

Tabla 3. Registros consultados conjuntamente al incorporar 2024.

Año	Yellow	Green	Total
2024	41,169,720	660,218	41,829,938
2026	29,703,355	337,114	30,040,469

Parquet contra tabla DuckDB

La tabla se materializó en 58.2 s y cada combinación se ejecutó dos veces. La Tabla 4 y la Figura 2 muestran la mediana de las ejecuciones.

Tabla 4. Mediana del tiempo de ejecución en segundos.

Escenario	Consulta	Parquet	Tabla DuckDB	Parquet / DuckDB
2024 (41.8 M)	Resumen mensual	2.763	1.028	2.69
2024 (41.8 M)	Distribución de pagos	0.867	0.686	1.26
2024 (41.8 M)	Rutas principales	0.934	0.752	1.24
2026 (30.0 M)	Resumen mensual	1.284	0.624	2.06
2026 (30.0 M)	Distribución de pagos	0.626	0.535	1.17
2026 (30.0 M)	Rutas principales	0.765	0.625	1.22
2024+2026 (71.9 M)	Resumen mensual	2.840	1.841	1.54
2024+2026 (71.9 M)	Distribución de pagos	1.370	1.325	1.03
2024+2026 (71.9 M)	Rutas principales	2.001	1.640	1.22

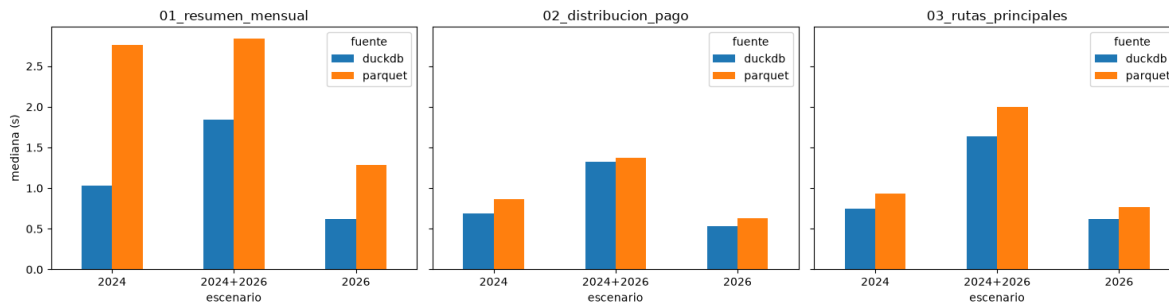


Figura 2. Mediana del tiempo de cada consulta según la fuente y el escenario.

La tabla fue más rápida en todos los casos, pero la ventaja dependió de la consulta: entre 1.5 y 2.7 veces en el resumen mensual y casi nada en la distribución de pagos. Tampoco creció con el volumen: con 2024 y 2026 juntos la razón fue menor que con un solo año.

Indicadores en Metabase

La Tabla 5 resume las diez preguntas que respondimos con indicadores. Las figuras siguientes son capturas del tablero de Metabase ya con los tres años.

Tabla 5. Preguntas e indicadores del tablero.

#	Pregunta	Indicador
1	¿Llevar más pasajeros aumenta el tiempo de viaje?	Duración mediana y minutos por milla por pasajeros
2	¿Cuánto dura un viaje y cómo cambia en el tiempo?	Duración mediana por mes y servicio
3	¿Con qué forma de pago la tarifa base es más alta?	Tarifa base media por forma de pago y año
4	¿Los viajes por plataforma tienen tarifa más alta?	Tarifa base y tarifa por milla medianas por origen
5	¿La propina cambia con la distancia?	Propina como % de la tarifa por rango de distancia
6	¿Qué tipo de tarifa se usa más y cuál es más cara?	% de viajes y tarifa media por tipo de tarifa
7	¿Hay destinos con propina más alta?	% de propina en las 10 zonas más altas y más bajas
8	¿Cómo evoluciona la demanda?	Viajes por día por mes y servicio
9	¿En qué horas y días se concentra la demanda?	% de viajes de cada día por hora
10	¿Cuántos viajes pagan el peaje de congestión?	% de viajes con cargo y recaudación mensual

En la Figura 3, la duración mediana sube de 12.6 minutos con un pasajero a 14.6 con cuatro, pero los minutos por milla bajan de 4.89 a 4.60: los grupos no viajan más lento, hacen viajes más largos (la distancia media pasa de 3.4 a 4.2 millas).

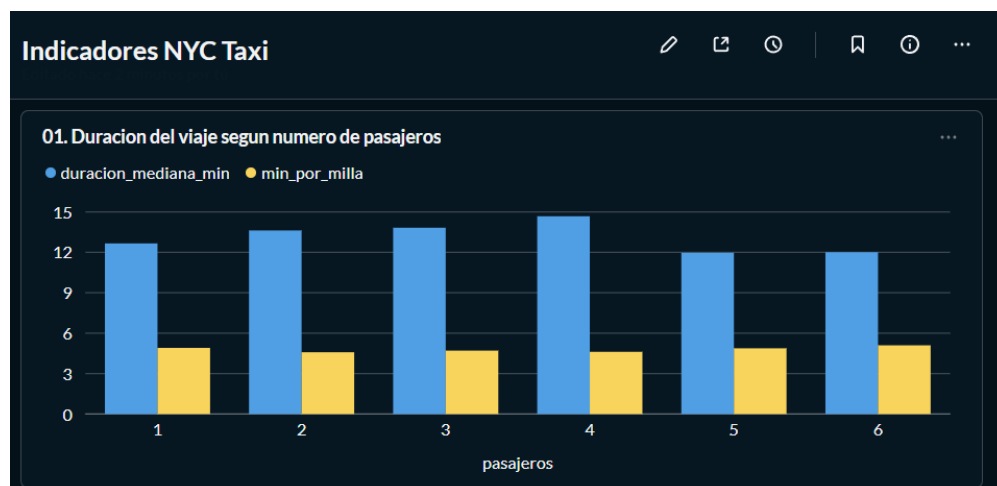


Figura 3. Indicador 1 en Metabase: duración según el número de pasajeros.

La Figura 4 muestra que la duración mediana crece en los tres años y que en 2024 y 2025 sus máximos están entre septiembre y diciembre.

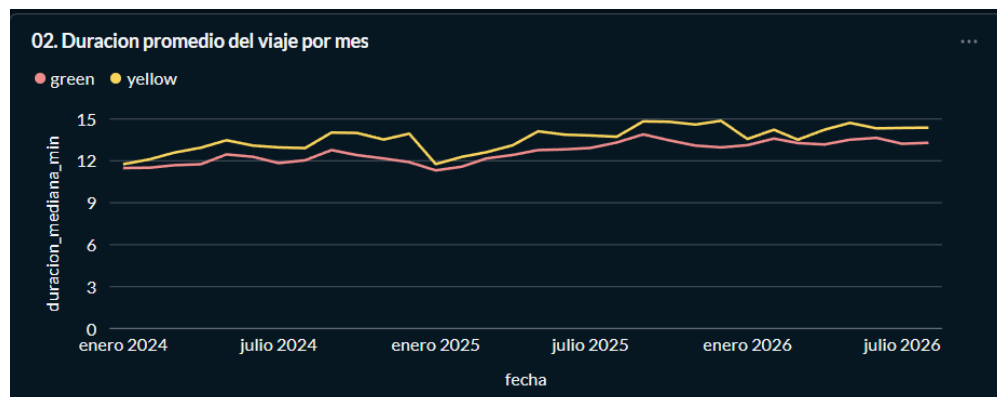


Figura 4. Indicador 2 en Metabase: duración mediana del viaje por mes.

En la Figura 5, tarjeta y efectivo tienen prácticamente la misma tarifa base (medianas de USD 12.80 a 13.50), así que la forma de pago no determina el precio. La más alta está en el código 0, que agrupa registros sin datos del taxímetro.

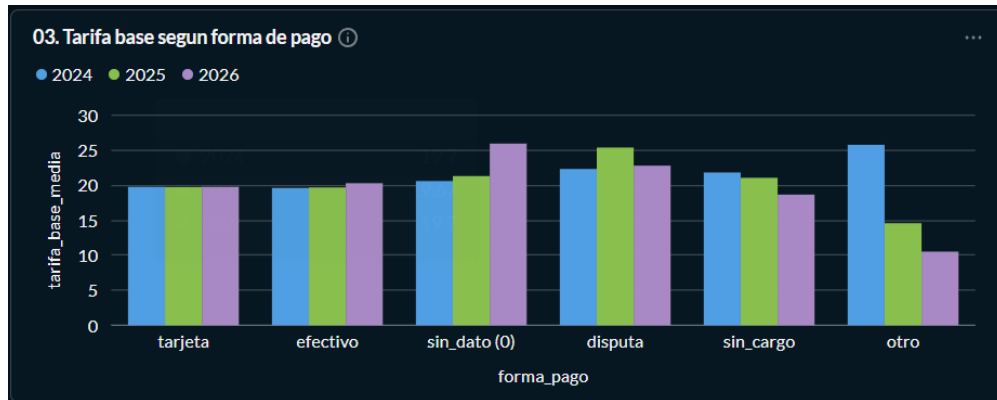


Figura 5. Indicador 3 en Metabase: tarifa base según forma de pago.

La Figura 6 responde a la pregunta sobre plataformas. Entre junio y agosto de 2026, el 26 % de los viajes Yellow llegó por plataforma. Su tarifa base mediana es mayor (Uber USD 22.98 contra USD 13.50 sin plataforma), pero porque son viajes más largos: la tarifa por milla de Uber (USD 7.17) es incluso algo menor que la de los viajes sin plataforma (USD 7.53).

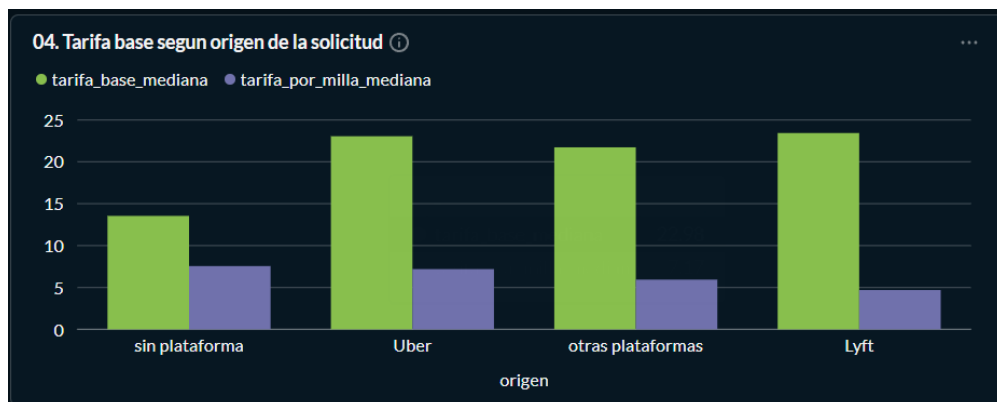


Figura 6. Indicador 4 en Metabase: tarifa base según el origen de la solicitud.

La Figura 7 muestra que la propina proporcional baja con la distancia, de cerca de 29 % en viajes de menos de una milla a 16 o 17 % en los de más de veinte. En la Figura 8, la tarifa estándar cubre más del 90 % de los viajes, JFK se mantiene cerca de su tarifa fija de USD 70 y Nassau/Westchester es la más cara.

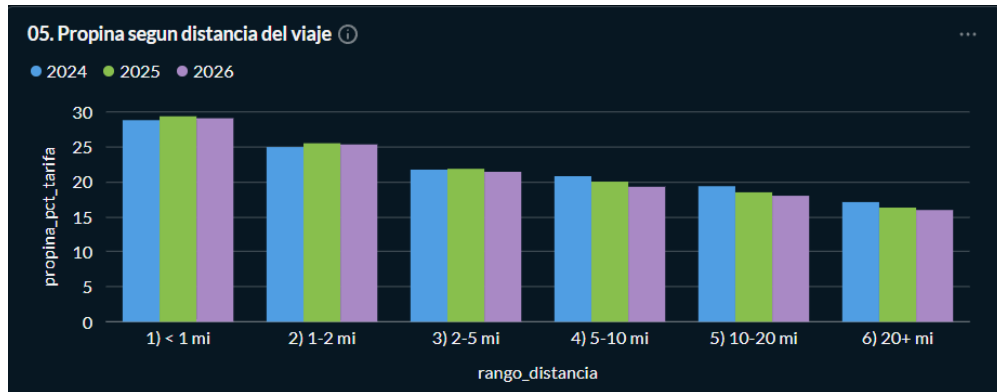


Figura 7. Indicador 5 en Metabase: propina según la distancia del viaje.

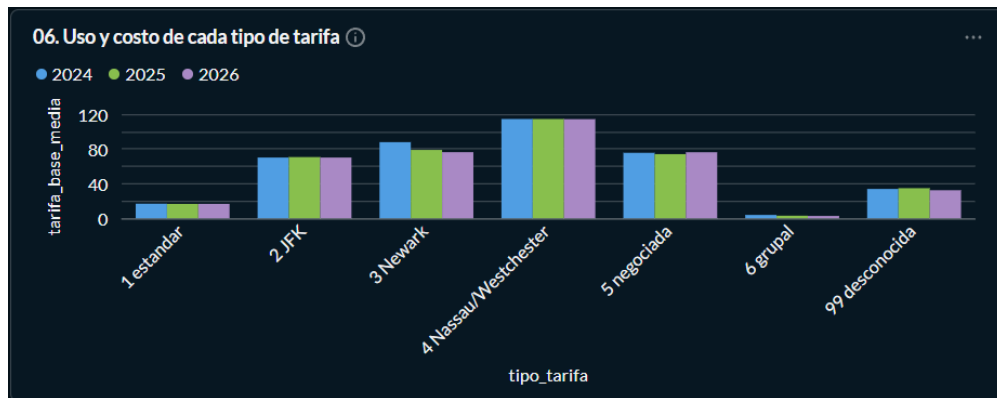


Figura 8. Indicador 6 en Metabase: tarifa base media por tipo de tarifa.

Para el indicador 7 usamos la gráfica del notebook (Figura 9), porque en la captura de Metabase las zonas con menor propina quedaron agrupadas en una sola barra. Las diez zonas con mayor propina están en Manhattan, con 24 a 25 % de la tarifa, mientras que en destinos de Brooklyn, Bronx y Queens como Starrett City (2.0 %) o Brownsville (3.0 %) la propina registrada es varias veces menor.

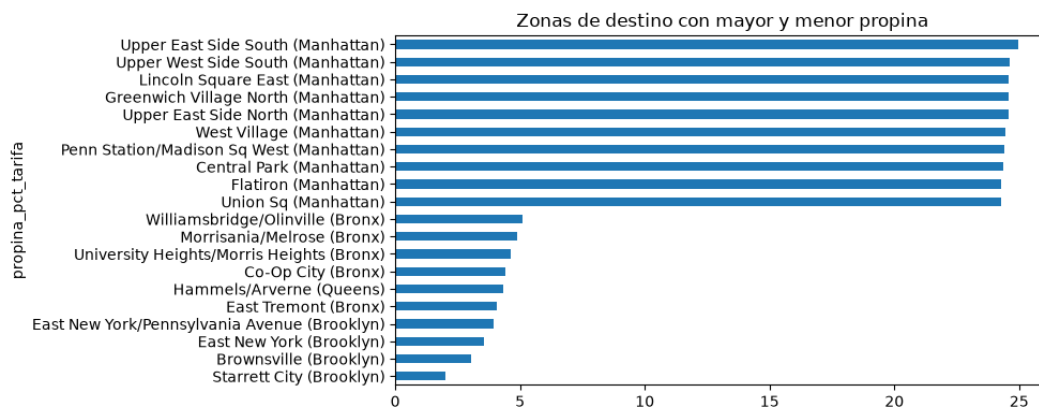


Figura 9. Indicador 7: zonas de destino con mayor y menor propina.

La Figura 10 muestra la demanda mensual. Green aparece casi plana porque su escala es cerca del 1 % de la de Yellow, pero en las tablas se ve que cae todos los años. La Figura 11 confirma el patrón por hora con los tres años: el domingo entre las 0 y las 2 horas ocurre el 15 % de sus viajes, frente al 2 % del martes.

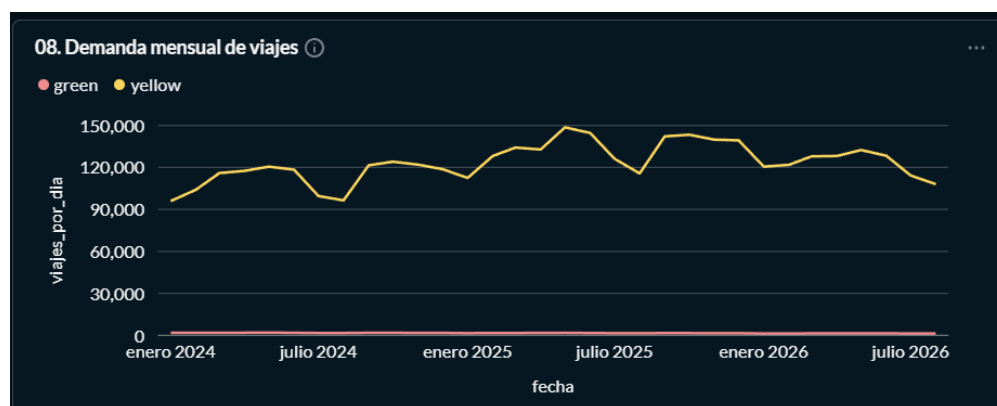


Figura 10. Indicador 8 en Metabase: viajes por día por mes y servicio.

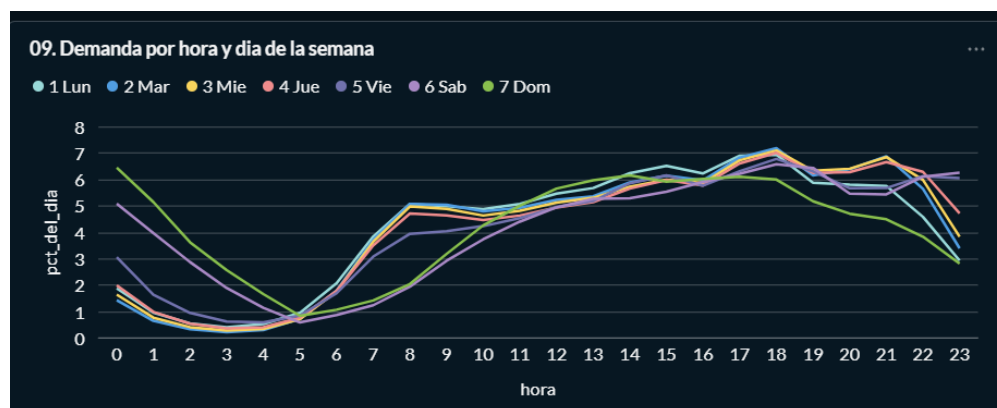


Figura 11. Indicador 9 en Metabase: porcentaje de viajes de cada día por hora.

Por último, la Figura 12 es quizá la más clara del tablero: el peaje de congestión no existe en 2024, aparece en enero de 2025 y desde febrero lo paga de forma estable entre 71 y 73 % de los viajes Yellow, con una recaudación de USD 1.7 a 2.4 millones por mes. En Green solo lo paga entre 7 y 10 %.

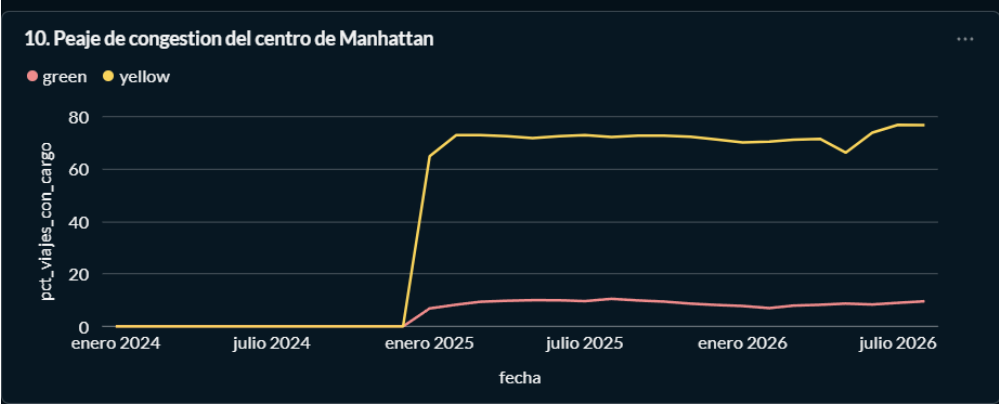


Figura 12. Indicador 10 en Metabase: porcentaje de viajes con peaje de congestión.

Evolución 2024 a 2026

Como 2026 solo tiene enero a agosto, comparamos los tres años en esos mismos meses. La Figura 13 resume las métricas principales.

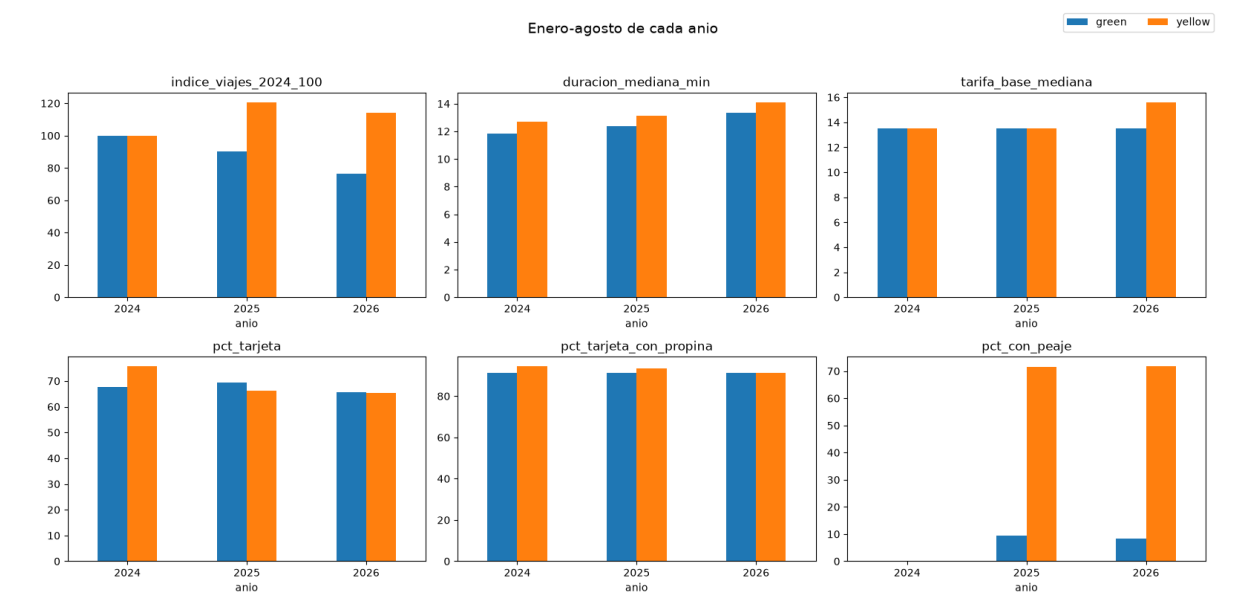


Figura 13. Métricas de enero a agosto de cada año por servicio.

A primera vista Yellow tuvo su máximo en 2025 (de 107 mil a 129 mil viajes por día, 21 % más) y la duración y la tarifa subían cada año. Pero al separar por tipo de registro encontramos que los viajes sin datos del taxímetro (código de pago 0) se triplicaron, como muestra la Tabla 6.

Tabla 6. Viajes Yellow por día según el tipo de registro, enero a agosto.

Año	Con taxímetro	Sin taxímetro (código 0)	% sin taxímetro
2024	97,051	10,302	9.53

Año	Con taxímetro	Sin taxímetro (código 0)	% sin taxímetro
2025	99,233	30,221	23.27
2026	90,480	31,756	25.98

Con viajes de tarifa estándar, que no se ven afectados por esa mezcla, la imagen cambia (Tabla 7): la tarifa mediana es la misma en los tres años, la velocidad es igual en 2024 y 2025 y solo baja en 2026, y entre los pagos conocidos la tarjeta gana terreno frente al efectivo.

Tabla 7. Control por composición en Yellow, enero a agosto.

Año	Tarifa mediana estándar (USD)	USD por milla	Velocidad mediana (mph)	% tarjeta frente a efectivo
2024	12.80	7.42	9.23	84.63
2025	12.80	7.45	9.18	87.29
2026	12.80	7.75	8.66	87.82

5. Discusión

El resultado que más nos hizo replantear el análisis fue el de la composición de los datos. Con dos años y métricas agregadas habríamos concluido que la demanda de Yellow creció 21 %, que la tarifa subió y que la gente pagaba menos con tarjeta. Al agregar 2025 y separar por tipo de registro, vimos que los viajes con taxímetro solo crecieron 2 % en 2025 y cayeron 9 % en 2026, que la tarifa estándar no cambió y que la tarjeta en realidad ganó terreno. Lo que cambió fue la forma de registrar los viajes: cada año hay más registros sin datos del taxímetro, que son viajes más largos e incluyen los pedidos por plataforma. Lo que sí parece real es que en 2026 los viajes van más lento (la velocidad estándar bajó 6 %) y que cada año menos viajes largos registran propina.

El peaje de congestión funcionó como un antes y un después: aparece de un mes a otro en enero de 2025 y se mantiene estable. Coincide con el año de mayor demanda total de Yellow, pero nuestros datos no permiten decir que una cosa causó la otra; para eso habría que comparar con los viajes de plataformas como servicio aparte, que la TLC publica en otros archivos.

Parquet o tabla materializada

Consultar Parquet directamente nos sirvió para explorar: no hay paso de carga, un mes nuevo aparece en la siguiente consulta y DuckDB solo lee las columnas que necesita. Su costo es volver

a leer y normalizar los archivos en cada consulta, y manejar las diferencias de esquema entre servicios y años, como `cbd_congestion_fee` en 2025 y `request_source` en 2026. La tabla materializada fue entre 1.03 y 2.7 veces más rápida y nos dio un esquema único, ideal para Metabase, pero cuesta tiempo y espacio, se desactualiza y solo admite un proceso escribiendo, así que hay que detener Metabase para reconstruirla. Nuestra conclusión práctica fue explorar sobre Parquet y materializar cuando las mismas consultas se repiten muchas veces.

Frente a Pandas

Con Pandas habría que cargar todo en memoria antes de calcular. Las 17 columnas de la tabla de viajes para 121 millones de filas ocuparían bastante más que los 8 GB del contenedor. DuckDB procesa por partes, lee solo las columnas que usa y, con un límite de memoria, usa disco temporal. En nuestro flujo Pandas solo recibe el resultado final, que son pocas filas, y ahí sí es cómodo para graficar.

Diseño para nuevos datos y reproducibilidad

Lo que permitió agregar 2025 cambiando una sola línea fue que las rutas usan comodines, el año sale del nombre del archivo y todas las métricas agrupan por año, `union_by_name` tolera columnas nuevas y las consultas viven en archivos SQL de los que salen la documentación, los notebooks y el tablero. Para mantener el proyecto reproducible fueron clave Docker con versiones fijas, no subir los datos a Git pero sí el script que los obtiene de la fuente, scripts que se pueden ejecutar varias veces sin efectos raros y tratar la base DuckDB como un derivado que siempre se puede reconstruir.

En un sistema de producción automatizaríamos la revisión mensual de publicaciones de la TLC, la validación, la reconstrucción de la base y la actualización del tablero, además de alertas sobre cambios de esquema y sobre indicadores de calidad como el porcentaje de registros sin taxímetro o con tarifa desconocida (que pasó de 1.2 % a 3.6 %). Sin esos avisos, un cambio en el registro puede distorsionar el análisis sin que nadie lo note.

Limitaciones

2026 solo tiene enero a agosto, por lo que las comparaciones anuales se hicieron en ese periodo. `request_source` solo existe desde junio de 2026, así que el análisis por plataforma cubre tres meses. Las propinas solo son confiables en pagos con tarjeta, y la mayoría de nuestras

explicaciones sobre causas (por ejemplo, la relación entre el peaje y la demanda) quedan como hipótesis que estos datos no permiten probar.

6. Conclusiones

7. DuckDB nos permitió analizar más de 121 millones de viajes con SQL dentro de un contenedor de 8 GB, consultando los Parquet directamente y sin cargar los datos en memoria.
8. Consultar Parquet es la mejor opción para explorar y para datos que llegan como archivos; materializar una tabla conviene cuando las consultas se repiten, como en un tablero. En nuestro benchmark la tabla fue entre 1.03 y 2.7 veces más rápida, con un costo de construcción de cerca de un minuto.
9. Diseñar el flujo con comodines, el año en el nombre del archivo y consultas versionadas hizo que incorporar 2025 requiriera cambiar una sola línea y ninguna consulta de indicadores.
10. Los diez indicadores mostraron patrones claros: el peaje de congestión alcanza a cerca del 72 % de los viajes Yellow desde 2025, la propina depende mucho del destino y Green pierde viajes todos los años.
11. La lección más importante fue revisar la composición de los datos antes de interpretar tendencias. El crecimiento de los registros sin taxímetro, de 9.5 % a 26 %, hacía parecer cambios de demanda, precio y forma de pago que en buena parte no existían.

7. Referencias

NYC Taxi and Limousine Commission. (2026). *TLC Trip Record Data*.

<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

NYC Taxi and Limousine Commission. *Data Dictionary: Yellow Taxi Trip Records*.

https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf

DuckDB Foundation. *DuckDB Documentation*. <https://duckdb.org/docs/>

The Apache Software Foundation. *Apache Parquet Documentation*.

<https://parquet.apache.org/docs/>

Metabase. *Metabase Documentation*. <https://www.metabase.com/docs/latest/>

MotherDuck. *metabase_duckdb_driver*.

https://github.com/motherduckdb/metabase_duckdb_driver

Metropolitan Transportation Authority. *Congestion Relief Zone*.

<https://congestionreliefzone.mta.info/>